

CYBERSECURITY PORTFOLIO LAB

Wireshark Network Traffic Investigation

SOC Homelab

AUTHOR

Elvis Tang

ROLE FOCUS

SOC Analyst I

LAB PLATFORM

Mac host with Ubuntu and Windows virtual machines

NETWORK MODE

Shared NAT

Report Overview

This report documents a beginner SOC investigation conducted in a controlled homelab. The project uses Ubuntu as the analyst workstation and Windows as the endpoint. Both virtual machines connect through the Mac host's Shared NAT network. The investigation is designed to demonstrate a repeatable analyst workflow: generate traffic, capture it, apply focused filters, interpret the evidence, and report findings without making claims beyond the available data.

Objective

Capture and analyze representative network traffic between the lab systems and external services, then explain what the packets reveal about protocols, endpoints, connection behavior, and potential security relevance. The finished report will show practical familiarity with packet analysis, endpoint context, evidence handling, and concise SOC documentation.

Scope

- Mac host providing the virtualization platform and Shared NAT connectivity.
- Ubuntu virtual machine serving as the analyst system and packet capture workstation.
- Windows virtual machine serving as the endpoint and controlled source of test traffic.
- Traffic intentionally generated for lab analysis, including ICMP, DNS, TCP, HTTP, and TLS where available.
- Supporting endpoint and network evidence such as IP configuration, active connections, and scan results.

Success Criteria

- Confirm and document the network configuration of both virtual machines.
- Capture traffic using an appropriate Ubuntu interface and preserve the relevant evidence.
- Use display filters to isolate protocol activity and identify the communicating endpoints.
- Explain each screenshot in plain language and connect it to a SOC analyst task.
- Separate observed facts from interpretation and avoid unsupported conclusions.

Lab Architecture

The lab places both virtual machines on Shared NAT networking. This gives the systems internet access through the Mac host while keeping the exercise logically separated from direct bridged exposure to the physical local network. Communication visibility depends on the virtualization platform and the selected capture interface; a normal Ubuntu capture may show only traffic sent or received by Ubuntu.

MAC HOST	
<i>hosts both virtual machines and provides network translation</i>	
SHARED NAT VIRTUAL NETWORK	
<i>connects the analyst and endpoint systems</i>	
UBUNTU ANALYST VM Wireshark, tcpdump, Nmap, and networking tools	WINDOWS ENDPOINT VM Controlled workstation activity, PowerShell, and Event Viewer

Figure 1 Logical architecture of the SOC homelab

Environment Inventory

Component	Role	Confirmed configuration	Details to record
Mac host	Virtualization host	Hosts both virtual machines	Mac model, macOS version, virtualization software
Ubuntu VM	Analyst workstation	Shared NAT; enp0s1 at 192.168.64.3/24	Ubuntu release version, hostname, and Wireshark capture interface confirmation
Windows VM	Endpoint or target	Shared NAT; Ethernet at 192.168.64.4/24	Red Hat VirtIO adapter; gateway, DHCP, and IPv4 DNS at 192.168.64.1
Virtual network	Connectivity layer	Shared NAT; 192.168.64.0/24; gateway 192.168.64.1	DNS server and VM to VM reachability

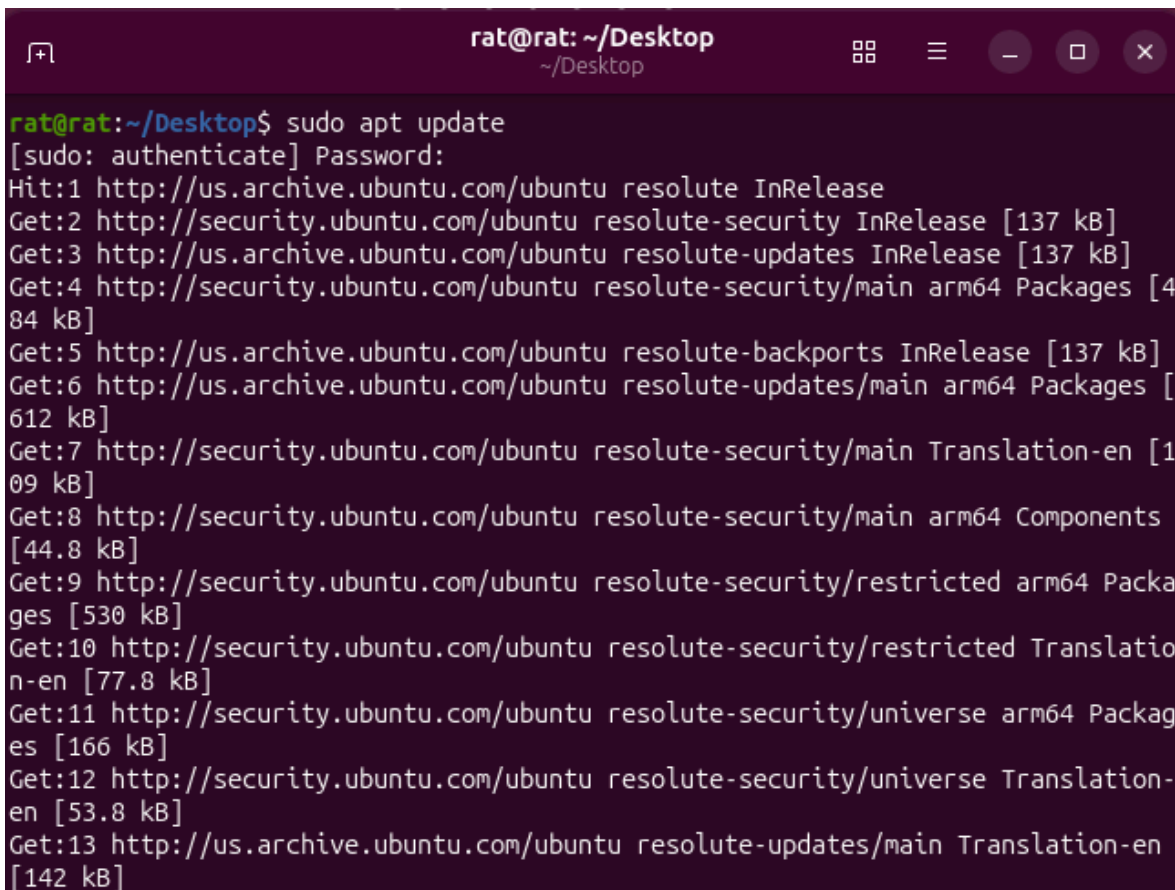
Tools and Analyst Use

The tools below support packet capture, traffic generation, endpoint validation, and evidence interpretation. Version numbers and installation details should be recorded from screenshots or command output when available.

System	Tool	Purpose in this investigation
Ubuntu	Wireshark	Capture traffic and inspect packet fields, conversations, timing, and protocol behavior.
Ubuntu	tcpdump	Validate traffic from the command line and create focused packet captures.
Ubuntu	Nmap	Test host reachability and identify exposed network services within the authorized lab scope.
Ubuntu	ip and ip route	Record interface addresses, network state, routes, and the default gateway.
Ubuntu	ping, nslookup, curl, ss	Generate or validate ICMP, DNS, application, and socket activity.
Windows	PowerShell	Generate controlled activity and inspect Windows network configuration and connections.
Windows	ipconfig and nslookup	Record addressing and DNS behavior on the endpoint.
Windows	netstat or Get-NetTCPConnection	Correlate active endpoint connections with captured packets.
Windows	Event Viewer	Review supporting Application, Security, and System logs when relevant evidence exists.

Ubuntu Analyst Workstation Preparation

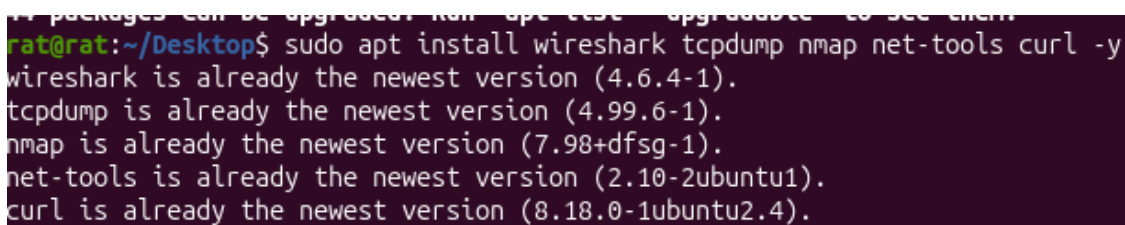
The following screenshots document repository access and the availability of the required analyst tools before packet analysis begins.



```
rat@rat: ~/Desktop
rat@rat:~/Desktop$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://us.archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://us.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main arm64 Packages [484 kB]
Get:5 http://us.archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:6 http://us.archive.ubuntu.com/ubuntu resolute-updates/main arm64 Packages [612 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [109 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/main arm64 Components [44.8 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/restricted arm64 Packages [530 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [77.8 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/universe arm64 Packages [166 kB]
Get:12 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [53.8 kB]
Get:13 http://us.archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [142 kB]
```

Figure 2 Ubuntu package index refresh using sudo apt update

Observation. The terminal is retrieving package metadata for the resolute release from us.archive.ubuntu.com and security.ubuntu.com. The visible downloads include arm64 package indexes. The screenshot captures the update while it is running and does not show the final completion message. **SOC relevance.** Repository reachability is a prerequisite for installing current defensive tools and security updates. Recording this step also makes the lab setup reproducible. **Finding.** The Ubuntu analyst VM could communicate with the configured Ubuntu repositories at the time shown. Final completion and upgrade status are not established by this screenshot.



```
rat@rat:~/Desktop$ sudo apt install wireshark tcpdump nmap net-tools curl -y
wireshark is already the newest version (4.6.4-1).
tcpdump is already the newest version (4.99.6-1).
nmap is already the newest version (7.98+dfsg-1).
net-tools is already the newest version (2.10-2ubuntu1).
curl is already the newest version (8.18.0-1ubuntu2.4).
```

Figure 3 Verification of required Ubuntu network analysis tools

Observation. The command `sudo apt install wireshark tcpdump nmap net-tools curl -y` reports that all five packages are already at their newest available versions. Visible versions are Wireshark 4.6.4-1, tcpdump 4.99.6-1, Nmap 7.98+dfsg-1, net-tools 2.10-2ubuntu1, and curl 8.18.0-1ubuntu2.4. **SOC relevance.** Documenting tool versions helps an analyst reproduce results and explain feature or output differences across lab environments. **Finding.** The required command-line and packet-analysis packages are installed. The screenshot does not by itself verify that Wireshark capture permissions or each tool's runtime behavior are correctly configured.

Core Wireshark Display Filters

```
icmp
dns
tcp
udp
http
tls
ip.addr == <observed host address>
tcp.port == 443
dns.qry.name contains "<observed domain>"
```

The angle-bracketed values are filter patterns, not recorded lab data. They must be replaced with values visible in the captured evidence.

Investigation Procedure

The procedure is organized to preserve a clear chain from test action to packet evidence. Each captured screenshot should show enough surrounding context to identify the system, interface or log source, filter or command, and relevant result.

Phase 1 Establish the Baseline

1. Start both virtual machines and confirm that each is attached to Shared NAT networking.
2. On Ubuntu, record the interface name, IPv4 address, route table, default gateway, and DNS configuration.
3. On Windows, record the adapter name, IPv4 address, subnet mask or prefix, default gateway, and DNS servers.
4. Test reachability between the virtual machines where the platform permits it, and document the result.

Phase 2 Prepare the Capture

5. Open Wireshark on Ubuntu and select the active Shared NAT interface based on observed traffic and interface addressing.
6. Begin a capture before generating test traffic so the request and response sequence is preserved.
7. Record the capture context, including the interface, approximate time, and activity being generated.

Phase 3 Generate and Isolate Traffic

8. Generate one controlled activity at a time, such as ping, DNS lookup, or a web request.
9. Apply a protocol or endpoint display filter to reduce unrelated background traffic.
10. Inspect packet details that explain the activity, including source and destination, protocol fields, flags, ports, response codes, and timing where visible.
11. Capture a screenshot that includes the display filter, packet list, and selected packet details whenever possible.

Phase 4 Correlate and Document

12. Compare packet evidence with the command or endpoint activity that generated it.
13. Write an observation limited to facts shown in the evidence.
14. Add an interpretation explaining what the fields mean and why they matter to a SOC analyst.
15. Record a finding only when the evidence supports a conclusion; otherwise state the limitation or next validation step.

Analyst Workflow

1 Generate	2 Capture	3 Filter	4 Interpret	5 Report
------------	-----------	----------	-------------	----------

Evidence Register

Each screenshot will receive a stable evidence ID. The register will be updated when an image is added, and the corresponding evidence section will contain the full caption, analyst interpretation, SOC relevance, and any supported finding.

ID	Evidence	Source	Status
E01	Ubuntu package index update	Ubuntu	Captured
E02	Analyst tool installation verification	Ubuntu	Captured
E03	Ubuntu interface configuration	Ubuntu	Captured
E04	Ubuntu routing table	Ubuntu	Captured
E05	Shared NAT gateway reachability	Ubuntu	Captured
E06	Windows IP configuration	Windows	Captured
E07	VM to VM reachability test	Ubuntu or Windows	Captured
E08	Wireshark ICMP echo request	Wireshark	Captured
E09	DNS lookup output	Ubuntu	Captured
E10	Wireshark TCP connection evidence	Wireshark	Captured
E11	HTTP application response	Ubuntu	Captured
E12	Active Windows connection evidence; see E28-E31	Windows	Captured
E13	Nmap scan attempt	Ubuntu	Inconclusive
E14	Windows Security audit event; see E34	Windows	Captured
E15	Local socket baseline	Ubuntu	Captured
E16	tcpdump packet capture	Ubuntu	Captured
E17	Wireshark DNS query and response	Wireshark	Captured
E18	Wireshark TCP and HTTP capture	Wireshark	Captured
E19	Shared NAT gateway service scan	Ubuntu	Captured
E20	Gateway scan and ping corroboration	Ubuntu	Captured
E21	Wireshark unfiltered capture overview	Wireshark	Captured
E22	Wireshark ICMP echo reply	Wireshark	Captured
E23	Wireshark HTTP GET request	Wireshark	Captured
E24	Wireshark HTTP 301 response	Wireshark	Captured
E25	Wireshark TLS 1.3 Client Hello	Wireshark	Captured
E26	Wireshark TLS 1.3 server response	Wireshark	Captured
E27	Wireshark TLS application data	Wireshark	Captured
E28	Windows TCP connection baseline	Windows	Captured
E29	Windows host-discovery scan	Ubuntu	Captured

ID	Evidence	Source	Status
E30	Windows -Pn scan in progress	Ubuntu	Captured
E31	Filtered established Windows connections	Windows	Captured
E32	Expanded established connection inventory	Windows	Captured
E33	Windows process lookup for captured PIDs	Windows	Captured
E34	Windows Security Event ID 4672	Windows	Captured
E35	Wireshark view of scan traffic targeting Windows	Wireshark	Captured
E36	Completed Nmap -Pn scan result for Windows	Ubuntu	Captured

Evidence Standards

- Preserve the visible command, filter, timestamp, packet fields, or log source needed to understand the screenshot.
- Redact personal, public IP, account, or device information before portfolio publication when appropriate.
- Use factual captions that identify what is visible; place interpretation in the analyst comment.
- Do not label activity malicious unless multiple evidence points support that conclusion.
- Record limitations such as partial captures, encrypted payloads, NAT visibility, or missing endpoint telemetry.

Screenshot Evidence and Analysis

This section is reserved for evidence supplied during the lab. Pending entries define what each screenshot should help establish; they do not represent completed observations. Images and comments will be inserted only after the visible evidence is reviewed.

Network Configuration Evidence

Evidence status: Reviewed - Ubuntu and Windows addressing documented; Windows-to-Ubuntu ICMP reachability captured.

Establish the addressing, routing, and network context needed to interpret later packet captures.

Evidence source	What the evidence establishes
Ubuntu ip addr output in Figure 4	Active interface enp0s1 with dynamic IPv4 address 192.168.64.3/24
Ubuntu ip route output in Figure 5	Default gateway 192.168.64.1 and connected route 192.168.64.0/24
Windows IP configuration output in Figure 12	Ethernet IPv4 address 192.168.64.4/24; gateway, DHCP, and IPv4 DNS 192.168.64.1
Windows ping output in Figure 13	Four replies from Ubuntu 192.168.64.3 with 0% packet loss

```
rat@rat:~/Desktop$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 26:17:c3:22:46:f1 brd ff:ff:ff:ff:ff:ff
    altname enx2617c32246f1
    inet 192.168.64.3/24 metric 1024 brd 192.168.64.255 scope global dynamic enp0s1
        valid_lft 3213sec preferred_lft 3213sec
    inet6 fd35:2cff:42cf:ebf9:2417:c3ff:fe22:46f1/64 scope global dynamic noprefixroute
        valid_lft 2591974sec preferred_lft 604774sec
    inet6 fe80::2417:c3ff:fe22:46f1/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

Figure 4 Ubuntu interface and address configuration from ip addr

Observation. The loopback interface and enp0s1 are present. enp0s1 is UP with MTU 1500 and has the dynamically assigned IPv4 address 192.168.64.3/24 with broadcast address 192.168.64.255. Global and link-local IPv6 addresses are also visible. **SOC relevance.** The interface name and analyst IP are essential for selecting the correct Wireshark capture interface and building host-specific display filters. **Finding.** The active Ubuntu network interface is enp0s1, and its observed IPv4 network is 192.168.64.0/24. The screenshot does not identify the Windows VM's address.

```

rat@rat:~/Desktop$ ip route
default via 192.168.64.1 dev enp0s1 proto dhcp src 192.168.64.3 metric 100
default via 192.168.64.1 dev enp0s1 proto dhcp src 192.168.64.3 metric 1024
192.168.64.0/24 dev enp0s1 proto kernel scope link src 192.168.64.3 metric 100
192.168.64.0/24 dev enp0s1 proto kernel scope link src 192.168.64.3 metric 1024
192.168.64.1 dev enp0s1 proto dhcp scope link src 192.168.64.3 metric 1024

```

Figure 5 Ubuntu routing table from ip route

Observation. The route table shows default routes through 192.168.64.1 on enp0s1 with source address 192.168.64.3 and metrics 100 and 1024. It also shows directly connected routes for 192.168.64.0/24. Linux normally prefers the lower metric route when otherwise equivalent routes exist. **SOC relevance.** Routing context helps an analyst determine whether a destination is local or reached through a gateway and prevents incorrect attribution during packet review. **Finding.** Traffic to destinations outside 192.168.64.0/24 is configured to use 192.168.64.1. Redundant route entries are visible, but this screenshot alone does not establish that they are causing a problem.

SOC analyst focus. A reliable baseline prevents analysts from misidentifying local systems, gateways, or external destinations during triage.

Analyst observation and finding: Ubuntu and Windows network identities are documented on the shared 192.168.64.0/24 subnet. Windows-to-Ubuntu reachability is confirmed in command output; packet-level correlation and reverse-direction testing remain pending.

ICMP Traffic Evidence

Evidence status: Reviewed - gateway and Windows-to-Ubuntu command output captured; Wireshark confirms the gateway ICMP request and reply sequence.

Document a controlled ping and the corresponding ICMP request and response packets where visible.

Evidence source	What the evidence establishes
Ping output in Figure 6	The Shared NAT gateway 192.168.64.1 returned visible ICMP replies
Wireshark icmp filter in Figures 17 and 18	Four visible echo request and reply pairs between 192.168.64.3 and 192.168.64.1
Windows ping output in Figure 13	Windows reached Ubuntu 192.168.64.3: 4 sent, 4 received, 0% loss

```

rat@rat:~/Desktop$ ping 192.168.64.1
PING 192.168.64.1 (192.168.64.1) 56(84) bytes of data.
64 bytes from 192.168.64.1: icmp_seq=1 ttl=64 time=0.408 ms
64 bytes from 192.168.64.1: icmp_seq=2 ttl=64 time=0.581 ms
64 bytes from 192.168.64.1: icmp_seq=3 ttl=64 time=0.560 ms
64 bytes from 192.168.64.1: icmp_seq=4 ttl=64 time=0.566 ms
64 bytes from 192.168.64.1: icmp_seq=5 ttl=64 time=0.621 ms
64 bytes from 192.168.64.1: icmp_seq=6 ttl=64 time=0.444 ms
64 bytes from 192.168.64.1: icmp_seq=7 ttl=64 time=0.623 ms
64 bytes from 192.168.64.1: icmp_seq=8 ttl=64 time=0.650 ms
64 bytes from 192.168.64.1: icmp_seq=9 ttl=64 time=0.686 ms
64 bytes from 192.168.64.1: icmp_seq=10 ttl=64 time=0.453 ms
64 bytes from 192.168.64.1: icmp_seq=11 ttl=64 time=0.413 ms
64 bytes from 192.168.64.1: icmp_seq=12 ttl=64 time=0.611 ms
64 bytes from 192.168.64.1: icmp_seq=13 ttl=64 time=0.537 ms
64 bytes from 192.168.64.1: icmp_seq=14 ttl=64 time=0.325 ms
64 bytes from 192.168.64.1: icmp_seq=15 ttl=64 time=0.464 ms
64 bytes from 192.168.64.1: icmp_seq=16 ttl=64 time=0.695 ms
64 bytes from 192.168.64.1: icmp_seq=17 ttl=64 time=0.370 ms
64 bytes from 192.168.64.1: icmp_seq=18 ttl=64 time=0.786 ms
64 bytes from 192.168.64.1: icmp_seq=19 ttl=64 time=0.394 ms
64 bytes from 192.168.64.1: icmp_seq=20 ttl=64 time=0.907 ms
64 bytes from 192.168.64.1: icmp_seq=21 ttl=64 time=0.546 ms
64 bytes from 192.168.64.1: icmp_seq=22 ttl=64 time=0.755 ms
64 bytes from 192.168.64.1: icmp_seq=23 ttl=64 time=0.426 ms

```

Figure 6 ICMP reachability test to the Shared NAT gateway

Observation. The ping command targets 192.168.64.1 and shows repeated 64-byte replies. In the visible portion, TTL is 64 and round-trip times range from 0.325 ms to 0.907 ms. The capture does not include the final ping summary, so an overall packet-loss percentage cannot be calculated. **SOC relevance.** A gateway ping verifies local path availability and creates controlled ICMP traffic that can later be matched to Wireshark frames. **Finding.** The Shared NAT gateway is reachable from Ubuntu during the test. This does not confirm reachability to the Windows VM or prove that Wireshark captured the ICMP exchange.

SOC analyst focus. ICMP can support reachability testing, host discovery, troubleshooting, and detection of unusual reconnaissance patterns.

Analyst observation and finding: Gateway and Windows-to-Ubuntu reachability are confirmed in command output, and Figures 17 and 18 provide packet-level confirmation of the Ubuntu-to-gateway ICMP exchange. An Ubuntu-to-Windows test and Wireshark evidence for Windows-generated ICMP remain pending.

DNS Traffic Evidence

Evidence status: Reviewed - command-line resolution and packet-level DNS query and response evidence are documented.

Trace a domain lookup from query to response and identify the returned record information visible in the capture.

Capture target	What the review will establish
nslookup output in Figure 7	Queried names, local stub resolver, and returned IPv4 or IPv6 answers
Wireshark dns filter in Figures 19 and 20	Ubuntu and gateway direction, UDP port 53, A and AAAA query-response pairs

```

rat@rat:~/Desktop$ nslookup
> google.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   google.com
Address: 142.251.41.14
Name:   google.com
Address: 2607:f8b0:4007:803::200e
> elvistangprojects.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   elvistangprojects.com
Address: 185.199.108.153
Name:   elvistangprojects.com
Address: 185.199.110.153
Name:   elvistangprojects.com
Address: 185.199.111.153
Name:   elvistangprojects.com
Address: 185.199.109.153
>

```

Figure 7 Ubuntu DNS lookups using nslookup

Observation. The interactive nslookup session queries google.com and elvistangprojects.com through resolver 127.0.0.53 on port 53. Non-authoritative answers show google.com with IPv4 address 142.251.41.14 and IPv6 address 2607:f8b0:4007:803::200e. The second domain returns four IPv4 addresses: 185.199.108.153, 185.199.109.153, 185.199.110.153, and 185.199.111.153. **SOC relevance.** DNS results help analysts connect user or application activity to named destinations and identify the addresses that can be correlated with later connection evidence. The loopback resolver address indicates a local stub; the upstream resolver is not shown. **Finding.** Name resolution succeeded for both requested domains in the visible session. This command output does not establish DNS packet timing, upstream resolver identity, destination legitimacy, or Wireshark capture success.

SOC analyst focus. DNS frequently provides early indicators of user activity, malware infrastructure contact, misconfiguration, or failed resolution.

TCP HTTP and TLS Evidence

Evidence status: Reviewed - curl, focused HTTP, TCP, and TLS 1.3 metadata are documented; encrypted application content remains unavailable.

Show how a client establishes a connection and, when available, how application or encrypted-session metadata appears in the packet capture.

Capture target	What the review will establish
TCP-focused capture in Figure 21	Endpoint direction, port 80, flags, HTTP request and response rows, and connection closure
curl HTTP responses in Figure 8	Redirect response body and visible server or destination context
TLS filter in Figures 24 through 26	Client Hello SNI, server response metadata, endpoint direction, port 443, and encrypted application data
HTTP filter in Figures 22 and 23	GET request, exact redirect response, endpoint direction, ports, and visible curl headers

```

rat@rat:~/Desktop$ curl google.com
<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
rat@rat:~/Desktop$ curl elvistangprojects.com

<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx</center>
</body>
</html>

```

Figure 8 HTTP redirect responses returned to curl

Observation. The curl request to google.com returns an HTML body labeled 301 Moved and includes a link to <http://www.google.com/>. The request to elvistangprojects.com returns an HTML body labeled 301 Moved Permanently and identifies nginx in the page content. **SOC relevance.** Redirects explain why activity may continue to a different host or URL after an initial request. SOC analysts correlate the redirect target, response headers, DNS results, and follow-on connections before deciding whether the behavior is expected or suspicious. **Finding.** Both hosts returned application content consistent with HTTP redirects. Because the command did not display response headers or packet details, the screenshot does not establish the complete Location header, remote endpoint, TCP exchange, or any TLS session.

SOC analyst focus. SOC analysts use connection metadata to reconstruct activity even when encrypted payload content is unavailable.

Endpoint and Scan Evidence

Evidence status: Partial - Windows addressing, active TCP connections, limited process attribution, one Security audit event, and gateway/Windows scan evidence are documented; controlled Windows-generated application traffic remains pending.

Correlate network captures with Windows connection data, relevant event logs, or an authorized lab scan.

Capture target	What the review will establish
Windows connection output in Figures 27, 30, and 31	Local and remote endpoints, TCP state, HTTPS destination port, and owning PID where displayed
Nmap attempt in Figure 11	No host-discovery response from 192.168.1.10; target does not match the documented lab subnet
Windows Security event in Figure 33	Event ID 4672, timestamp, audit result, account context, logon ID, and visible assigned privileges
Windows IP configuration in Figure 12	Windows Ethernet addressing and Shared NAT gateway, DHCP, and DNS context
Windows ping in Figure 13	One-directional Windows-to-Ubuntu ICMP reachability and packet-loss summary
Gateway service scan in Figure 14	Responsive gateway and TCP ports reported open by Nmap without service-version verification
Combined gateway validation in Figure 15	Gateway reachability summary; repeated scan output is corroborating context, not an independent scan
Windows scan evidence in Figures 28, 29, 34, and 35	Discovery failure, -Pn connection attempts, packet-level SYN evidence, and a completed result reporting 1,000 filtered TCP ports with no response
Process lookup in Figure 32	PID 6268 resolves to Widgets and PID 3524 resolves to svchost at the time shown

Ubuntu Socket Baseline

```

rat@rat:~/Desktop$ ss -tuln
Netid  State  Recv-Q  Send-Q  Local Address:Port  Peer Address:Port
udp    UNCONN 0        0       127.0.0.1:323       0.0.0.0:*
udp    UNCONN 0        0       0.0.0.0:5353       0.0.0.0:*
udp    UNCONN 0        0       127.0.0.54:53      0.0.0.0:*
udp    UNCONN 0        0       127.0.0.53%lo:53   0.0.0.0:*
udp    UNCONN 0        0       192.168.64.3%enp0s1:68 0.0.0.0:*
udp    UNCONN 0        0       [::1]:323          [::]:*
udp    UNCONN 0        0       [::]:5353          [::]:*
tcp    LISTEN 0        4096    127.0.0.54:53      0.0.0.0:*
tcp    LISTEN 0        10      127.0.0.1:9843     0.0.0.0:*
tcp    LISTEN 0        4096    127.0.0.1:631      0.0.0.0:*
tcp    LISTEN 0        4096    127.0.0.53%lo:53   0.0.0.0:*
tcp    LISTEN 0        4096    [::1]:631         [::]:*

```

Figure 9 Ubuntu listening and bound sockets from ss

Observation. The ss -tuln output shows numeric TCP listeners and UDP sockets. Visible entries include loopback DNS sockets on 127.0.0.53 and 127.0.0.54 port 53, UDP port 5353 bound to all IPv4 and IPv6 addresses, DHCP client port 68 on 192.168.64.3%enp0s1, and loopback TCP listeners on ports 53, 631, and 9843. **SOC relevance.** A

Command Line Packet Capture

Figure 10 Command line packet capture on `enp0s1` using `tcpdump`

Nmap Scan Attempt

Figure 11 Nmap host discovery result for 192.168.1.10

Elvis Tang | Wireshark Network Traffic Investigation |

relevance. Analysts must verify the target address and authorized scope before interpreting negative scan results. Host-discovery failure can reflect an incorrect address, routing boundary, filtering, or an offline system. **Finding.** The scan received no host-discovery response, but 192.168.1.10 is outside the documented 192.168.64.0/24 virtual network. The screenshot therefore does not prove that the Windows VM is down. Confirm the Windows IPv4 address first and scan only that authorized target; use -Pn only after scope and addressing are verified.

Windows Endpoint Configuration



```

Administrator: Windows PowerShell
Windows IP Configuration

Host Name . . . . . : WIN-3HRELJRC5TH
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet:

Connection-specific DNS Suffix . :
Description . . . . . : Red Hat VirtIO Ethernet Adapter
Physical Address. . . . . : 0A-66-A0-9E-86-03
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . : Yes
IPv6 Address. . . . . : fd35:2cff:42cf:ebf9:3d66:30b1:c97c:301
3(Preferred)
Temporary IPv6 Address. . . . . : fd35:2cff:42cf:ebf9:4cd9:dc49:5e73:f91
f(Preferred)
Link-local IPv6 Address . . . . . : fe80::3b71:1a8c:fe82:ace6%6(Preferred)

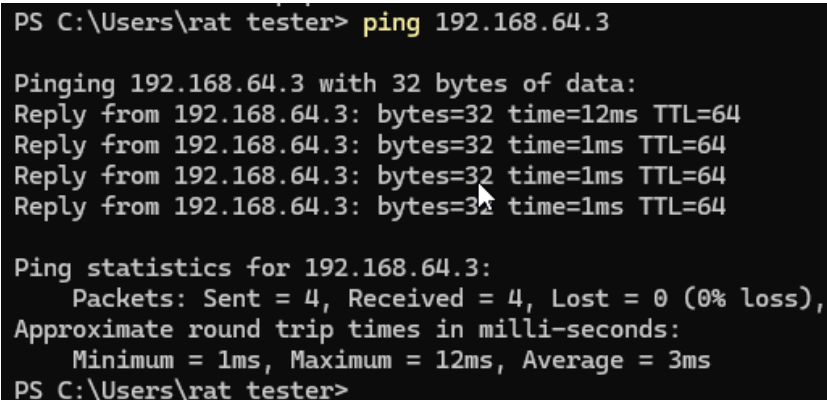
IPv4 Address. . . . . : 192.168.64.4(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Sunday, September 6, 2026 6:29:37 PM
Lease Expires . . . . . : Sunday, September 6, 2026 7:29:37 PM
Default Gateway . . . . . : 192.168.64.1
DHCP Server . . . . . : 192.168.64.1
DHCPv6 IAID . . . . . : 101344928
DHCPv6 Client DUID. . . . . : 00-01-00-01-32-29-3F-47-0A-66-A0-9E-86
-03
DNS Servers . . . . . : fe80::fcb2:14ff:fec6:9864%6
192.168.64.1
fe80::fcb2:14ff:fec6:9864%6
NetBIOS over Tcpip. . . . . : Enabled
PS C:\Users\rat tester>

```

Figure 12 Windows Ethernet addressing and Shared NAT configuration

Observation. The Windows Ethernet adapter is identified as Red Hat VirtIO. DHCP is enabled, the IPv4 address is 192.168.64.4 with subnet mask 255.255.255.0, and the default gateway, DHCP server, and IPv4 DNS server are 192.168.64.1. IPv6 addressing and DNS entries, a physical address, and NetBIOS over TCP/IP status are also visible. **SOC relevance.** Endpoint addressing establishes the host identity needed to correlate Windows activity with Ubuntu captures. The shared 192.168.64.0/24 prefix places Windows and Ubuntu on the same documented virtual subnet. **Finding.** The Windows endpoint is documented at 192.168.64.4/24 and uses the Shared NAT gateway 192.168.64.1 for gateway, DHCP, and IPv4 DNS functions. The screenshot does not show active connections, firewall policy, or process attribution.

Windows to Ubuntu Reachability



```
PS C:\Users\rat tester> ping 192.168.64.3

Pinging 192.168.64.3 with 32 bytes of data:
Reply from 192.168.64.3: bytes=32 time=12ms TTL=64
Reply from 192.168.64.3: bytes=32 time=1ms TTL=64
Reply from 192.168.64.3: bytes=32 time=1ms TTL=64
Reply from 192.168.64.3: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.64.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 12ms, Average = 3ms
PS C:\Users\rat tester>
```

Figure 13 Windows ICMP reachability test to Ubuntu 192.168.64.3

Observation. Windows sends four 32-byte ICMP echo requests to Ubuntu 192.168.64.3 and receives four replies. The summary reports 0% packet loss, TTL 64, and round-trip times of 1 ms minimum, 12 ms maximum, and 3 ms average. **SOC relevance.** A controlled ping confirms basic network-layer communication from the endpoint to the analyst workstation and provides a known event that can be correlated with packet captures. **Finding.** Windows-to-Ubuntu ICMP reachability succeeded during the visible test. This result does not establish Ubuntu-to-Windows reachability, application-port access, or Wireshark capture of the exchange.

Shared NAT Gateway Service Scan

```
rat@rat: ~/Desktop
~ / Desktop

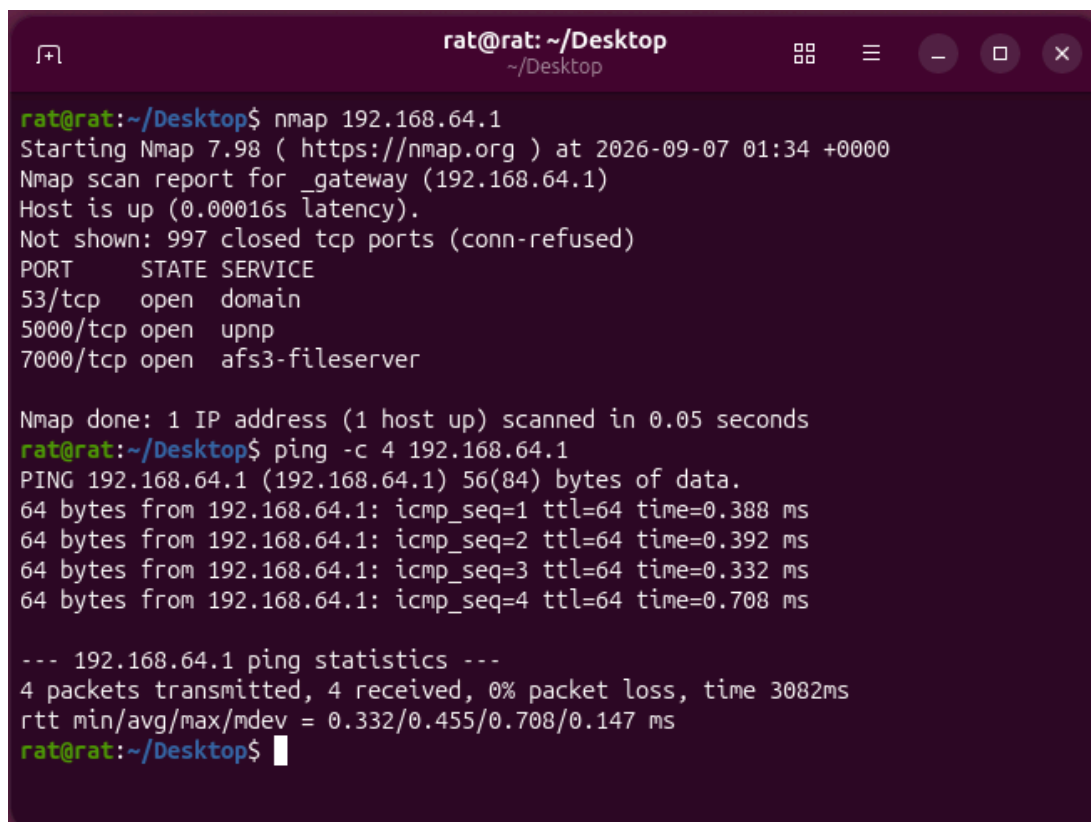
rat@rat:~/Desktop$ nmap 192.168.64.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-07 01:34 +0000
Nmap scan report for _gateway (192.168.64.1)
Host is up (0.00016s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE
53/tcp    open  domain
5000/tcp  open  upnp
7000/tcp  open  afs3-fileserver

Nmap done: 1 IP address (1 host up) scanned in 0.05 seconds
rat@rat:~/Desktop$
```

Figure 14 Nmap TCP scan of the Shared NAT gateway 192.168.64.1

Observation. Nmap scans 192.168.64.1 and reports the gateway host up with approximately 0.00016-second latency. Of the default 1,000 TCP ports, 997 are reported closed and ports 53, 5000, and 7000 are reported open with the labels domain, upnp, and afs3-fileserver. **SOC relevance.** A scoped service baseline helps analysts identify exposed ports and distinguish expected infrastructure from unexpected listening services. Nmap's service column in this output is based on conventional port mappings because no service-version detection result is shown. **Finding.** The Shared NAT gateway responded and Nmap reported TCP ports 53, 5000, and 7000 open. The screenshot does not verify the actual software behind those ports, and it does not describe services on the Windows endpoint.

Gateway Scan and Reachability Corroboration



```

rat@rat: ~/Desktop
~ / Desktop

rat@rat:~/Desktop$ nmap 192.168.64.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-07 01:34 +0000
Nmap scan report for _gateway (192.168.64.1)
Host is up (0.00016s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE
53/tcp    open  domain
5000/tcp  open  upnp
7000/tcp  open  afs3-fileserver

Nmap done: 1 IP address (1 host up) scanned in 0.05 seconds
rat@rat:~/Desktop$ ping -c 4 192.168.64.1
PING 192.168.64.1 (192.168.64.1) 56(84) bytes of data.
64 bytes from 192.168.64.1: icmp_seq=1 ttl=64 time=0.388 ms
64 bytes from 192.168.64.1: icmp_seq=2 ttl=64 time=0.392 ms
64 bytes from 192.168.64.1: icmp_seq=3 ttl=64 time=0.332 ms
64 bytes from 192.168.64.1: icmp_seq=4 ttl=64 time=0.708 ms

--- 192.168.64.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3082ms
rtt min/avg/max/mdev = 0.332/0.455/0.708/0.147 ms
rat@rat:~/Desktop$

```

Figure 15 Combined gateway scan and four-packet ICMP validation

Observation. The terminal repeats the gateway scan output and then runs a four-packet ping to 192.168.64.1. All four requests receive replies with 0% packet loss; the visible round-trip minimum, average, maximum, and deviation are 0.332, 0.455, 0.708, and 0.147 ms. **SOC relevance.** Combining service-discovery output with a finite reachability test records both port-scan context and a complete ICMP loss and latency summary for the infrastructure target. **Finding.** Gateway reachability is corroborated by the four successful ping replies. The repeated Nmap portion matches Figure 14 and should not be treated as an independent scan result or as evidence about the Windows VM.

SOC analyst focus. Cross-source correlation helps distinguish expected system behavior from activity that requires deeper investigation.

Analyst observation and finding: Ubuntu socket and command-line capture baselines, Windows addressing, Windows-to-Ubuntu reachability, a selected Windows Security audit event, and packet-level scan traffic are documented. The original scan of 192.168.1.10 remains inconclusive because it targeted outside the lab subnet. The gateway scan establishes only gateway exposure. Supplemental Windows evidence documents active TCP sessions and maps two owning PIDs to process names; two PIDs and controlled Windows-generated application traffic remain pending.

Wireshark Packet Capture Evidence

Evidence status: Reviewed - capture-interface context and filtered ICMP, DNS, TCP, HTTP, and TLS evidence are documented.

Validate packet capture on the Ubuntu analyst interface, reduce background traffic with focused display filters, and inspect controlled ICMP, DNS, TCP, HTTP, and TLS activity.

Unfiltered Capture Overview

The screenshot displays the Wireshark interface with the title bar 'Capturing from enp0s1'. The menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. The toolbar contains icons for common actions like opening files, saving, and filtering. The display filter bar shows 'Apply a display filter ... <Ctrl-/>'. The packet list pane shows a table of captured packets with columns for No., Time, Source, Destination, Protocol, and Length. The packet details pane shows the structure of the selected packet (Frame 1: Packet, 100 bytes on wire (800 bits), 100 bytes captured (800 bytes) on interface enp0s1). The packet structure is as follows:

Offset	Length	Protocol	Details
0000	14	Ethernet II	Src: 26:17:c3:22:46:f1 (26:17:c3:22:46:f1)
0010	20	Internet Protocol Version 4	Src: 192.168.64.3, Dst: 192.168.64.1
0020	28	User Datagram Protocol	Src Port: 39479, Dst Port: 53
0030	12	Domain Name System	Standard query query

Figure 16 Unfiltered Wireshark capture on Ubuntu interface enp0s1

Observation. Wireshark is actively capturing from enp0s1 with no display filter applied. The visible packet list includes NTP traffic involving Ubuntu 192.168.64.3 and public time servers, ICMPv6 control messages, mDNS traffic, and ARP activity for 192.168.64.1. The selected frame is decoded as Ethernet II, IPv4, UDP, and a DNS query, with 192.168.64.3 visible as the IPv4 source. **SOC relevance.** An unfiltered baseline confirms that the selected interface carries observable traffic and shows the background protocols an analyst must separate from the activity under investigation. **Finding.** Packet capture is functioning on enp0s1 and contains multiple routine network protocols. The screenshot does not by itself identify malicious activity or demonstrate Windows endpoint traffic.

Filtered ICMP Echo Request

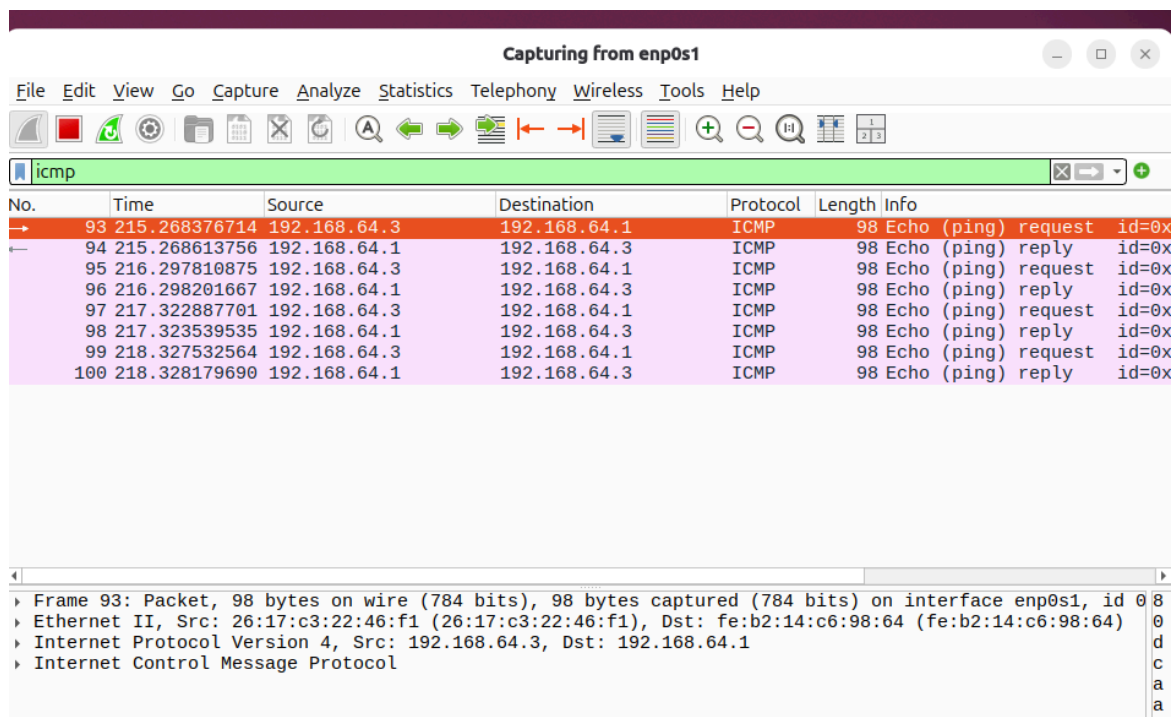


Figure 17 Wireshark ICMP filter with an Ubuntu echo request selected

Observation. The accepted display filter icmp isolates frames 93 through 100. The visible list contains four echo requests from 192.168.64.3 to 192.168.64.1 and four replies in the opposite direction. Selected frame 93 is a 98-byte ICMP echo request; the packet details show Ethernet II and IPv4 source and destination context from Ubuntu to the gateway. **SOC relevance.** A protocol filter reduces unrelated traffic and lets an analyst verify the direction, timing, and response pattern of a controlled connectivity test at the packet level. **Finding.** The capture records an ICMP echo request from the Ubuntu analyst VM to the Shared NAT gateway and shows a reply after each of the four visible requests. This view does not establish traffic outside the displayed packet set.

Filtered ICMP Echo Reply

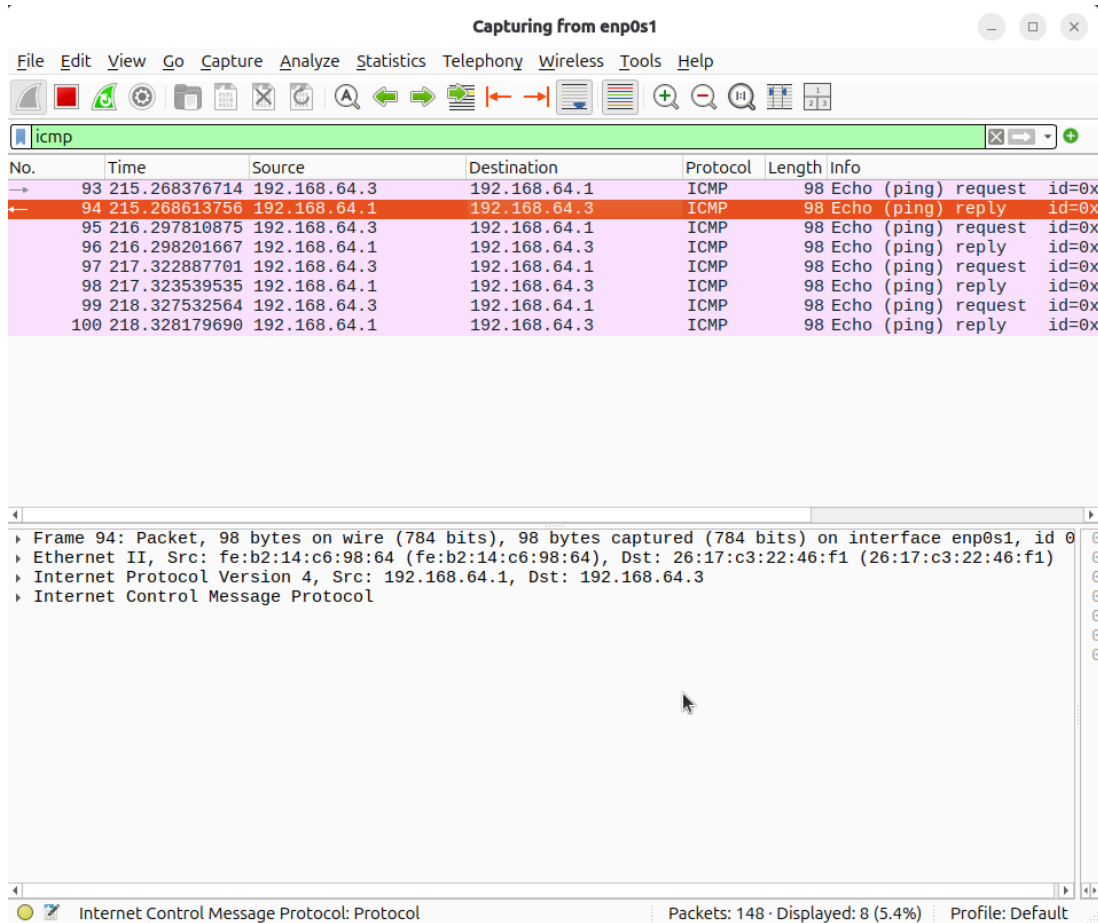


Figure 18 Wireshark ICMP filter with the gateway echo reply selected

Observation. Selected frame 94 is a 98-byte ICMP echo reply from 192.168.64.1 to 192.168.64.3. The packet list again shows four visible request and reply pairs, while the status bar reports 148 captured packets and 8 displayed packets, or 5.4 percent of the capture. **SOC relevance.** Inspecting the return direction corroborates that the gateway responded and demonstrates how a focused display filter can reduce a larger capture to the packets relevant to a specific question. **Finding.** The selected gateway response and the four visible request and reply pairs provide packet-level evidence of the Ubuntu-to-gateway ICMP exchange. The detailed ICMP identifier and sequence fields are not expanded in this screenshot, and Figures 17 and 18 are companion views of the same filtered capture rather than separate tests.

Filtered DNS Query

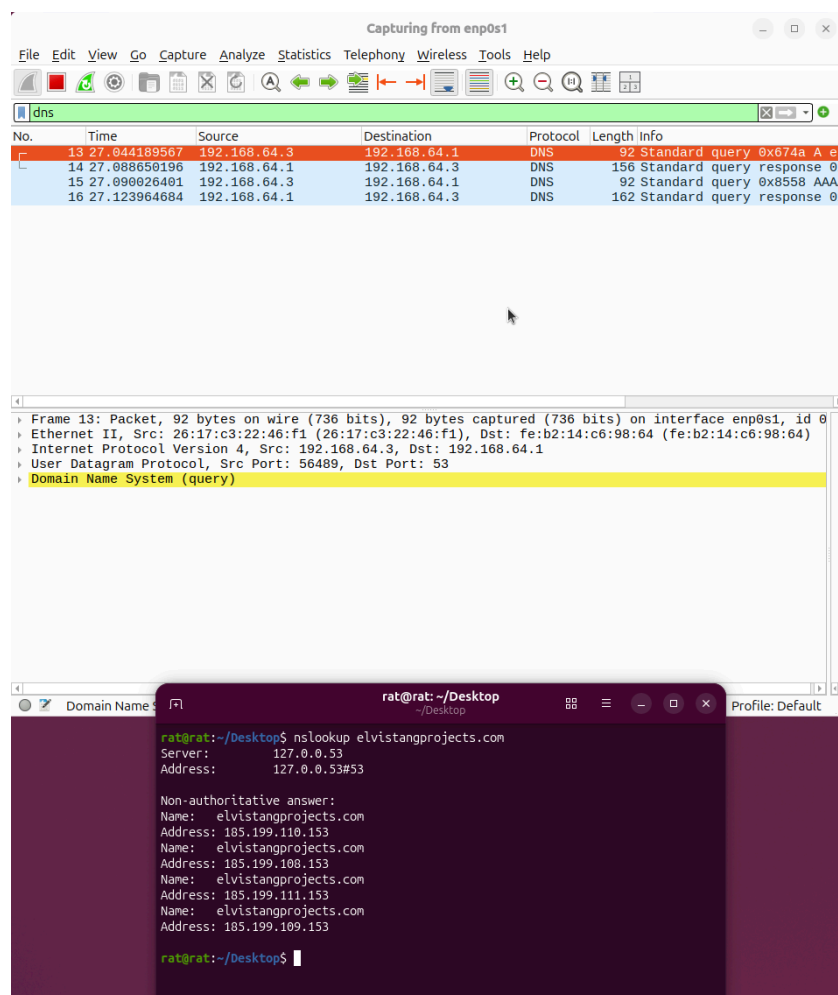


Figure 19 Wireshark DNS filter with the client query selected

Observation. The accepted dns display filter shows four packets: an A query and response followed by an AAAA query and response between Ubuntu 192.168.64.3 and gateway 192.168.64.1. Selected frame 13 is a 92-byte client-to-resolver packet using UDP source port 56489 and destination port 53. The visible nslookup output for elvistanprojects.com lists IPv4 answers 185.199.110.153, 185.199.108.153, 185.199.111.153, and 185.199.109.153. **SOC relevance.** DNS telemetry helps connect a user command to name-resolution activity, distinguish the client and resolver roles, and identify returned addresses for correlation with later connections. **Finding.** A controlled lookup produced visible A and AAAA DNS exchanges between Ubuntu and the Shared NAT gateway. The packet details are not expanded far enough to verify individual answer records inside the selected Wireshark frame, and the upstream resolver is not shown.

Filtered DNS Response

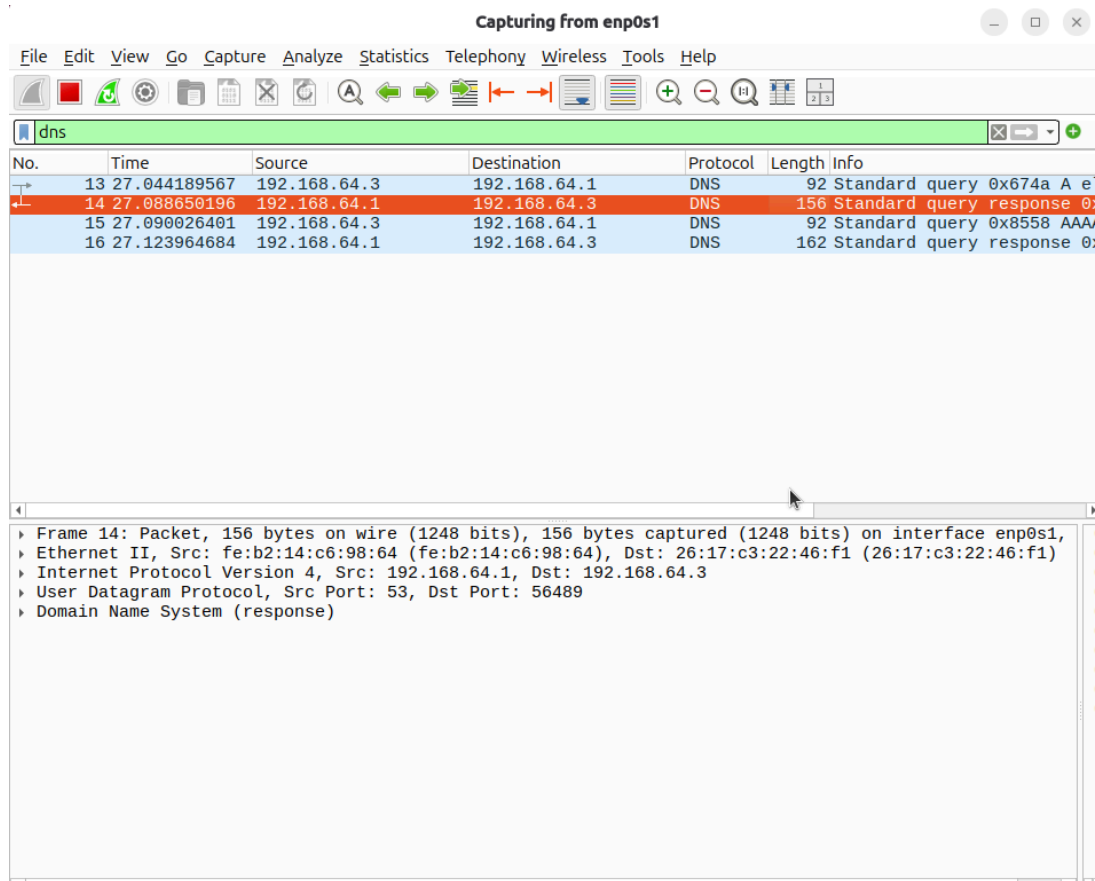


Figure 20 Wireshark DNS filter with the resolver response selected

Observation. Selected frame 14 is a 156-byte DNS response from 192.168.64.1 to 192.168.64.3. The packet details show UDP source port 53 and destination port 56489, reversing the transport direction of the selected query in Figure 19. The list also shows a second query-response pair associated with an AAAA request. **SOC relevance.** Reviewing both directions lets an analyst confirm that a request received a response and correlate the return traffic with the initiating client port. **Finding.** The captured resolver response corresponds directionally and by port to the selected client query. The DNS tree is not expanded, so response code, answer count, TTL values, and individual resource records are not verified from this screenshot.

Filtered TCP and HTTP Activity

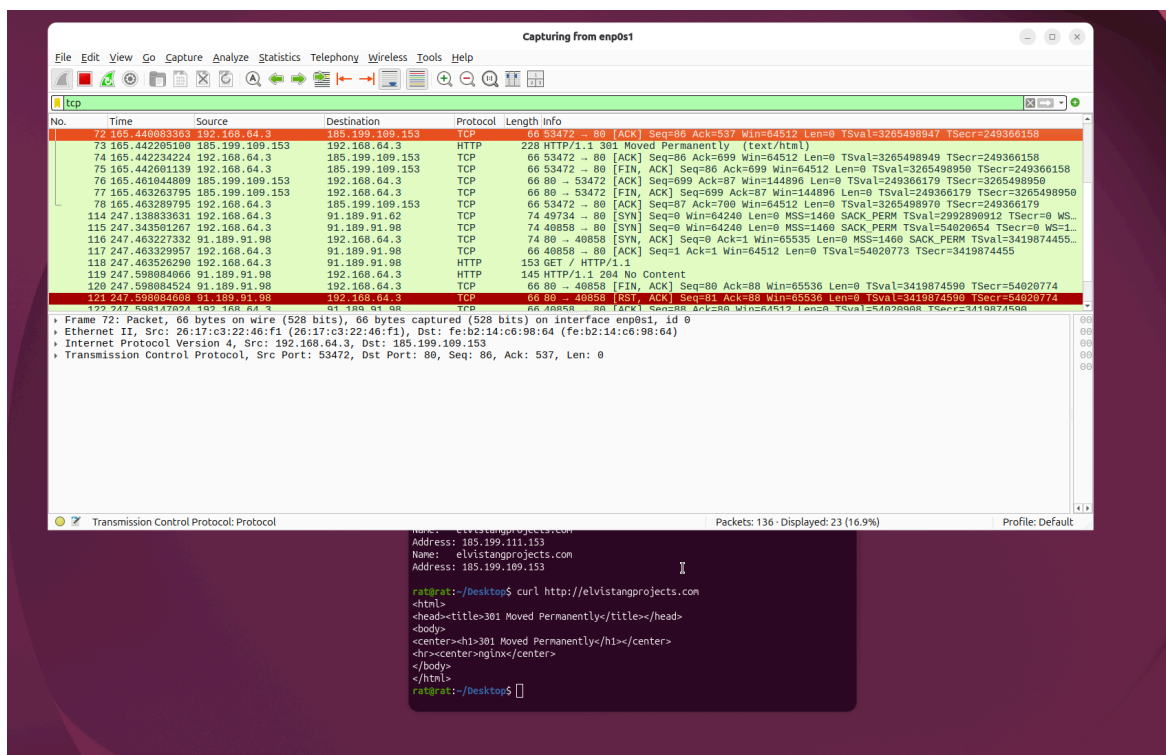


Figure 21 Wireshark TCP filter with HTTP redirect activity

Observation. The tcp filter displays a conversation between Ubuntu 192.168.64.3 and 185.199.109.153 on TCP port 80. Selected frame 72 is a 66-byte acknowledgment from source port 53472 to destination port 80, and the next visible row is labeled HTTP/1.1 301 Moved Permanently. A separate visible port 80 exchange with 91.189.91.98 includes SYN, SYN-ACK, ACK, an HTTP GET, an HTTP/1.1 204 No Content response, and connection-close packets. The terminal shows curl returning a 301 Moved Permanently page for elvistanprojects.com. **SOC relevance.** Packet sequencing and flags let an analyst reconstruct connection establishment and teardown, while HTTP dissection connects transport activity to the application response seen by the user. **Finding.** The capture corroborates plaintext HTTP activity and a 301 redirect response associated with the controlled curl test. Because the HTTP headers and payload are not expanded, the complete Location header and any later redirected or TLS session cannot be confirmed from this view. No malicious conclusion is supported by the screenshot alone.

Filtered HTTP GET Request

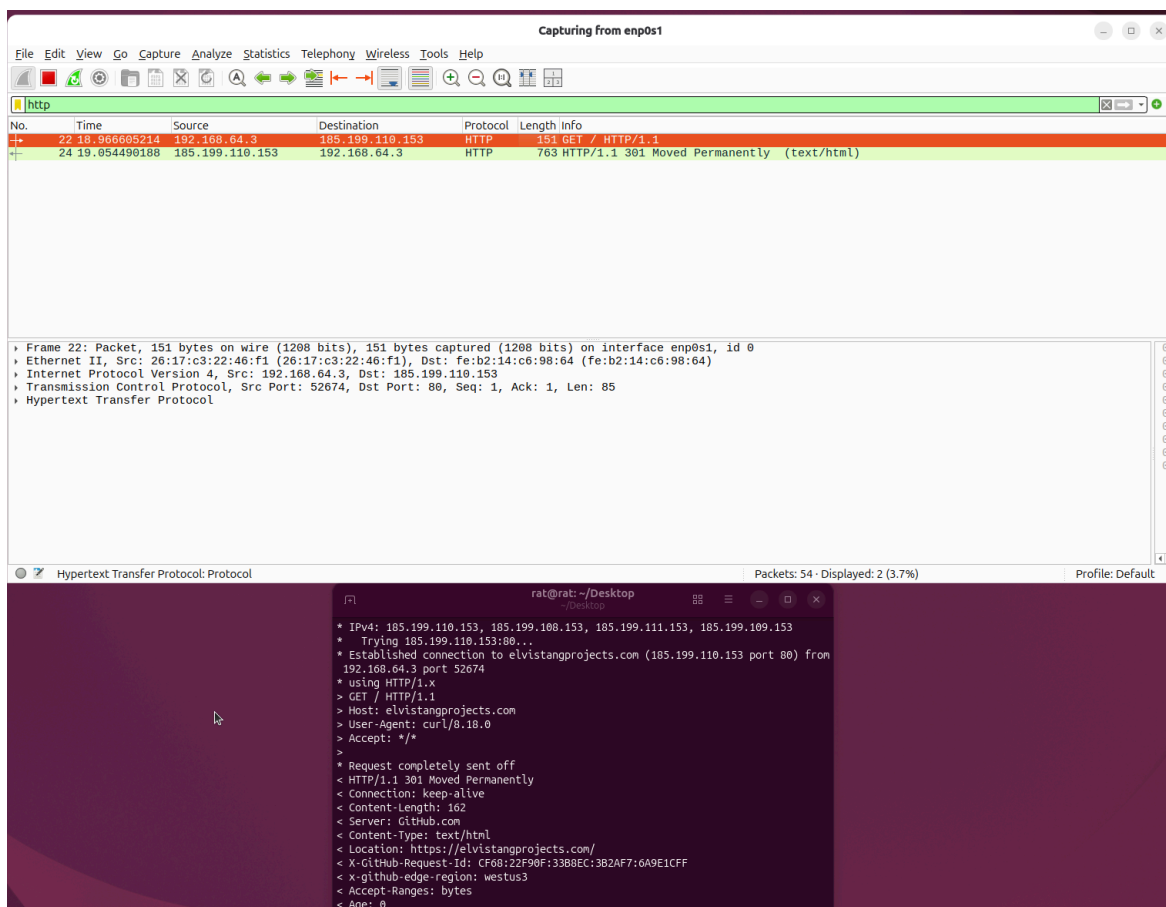


Figure 22 Wireshark HTTP filter with the GET request selected

Observation. The accepted http filter isolates two packets. Selected frame 22 is a 151-byte GET / HTTP/1.1 request from Ubuntu 192.168.64.3 to 185.199.110.153. Packet details show TCP source port 52674, destination port 80, sequence 1, acknowledgment 1, and 85 bytes of TCP payload. The visible curl output records a connection to elvistingprojects.com at 185.199.110.153:80 and shows Host: elvistingprojects.com, User-Agent: curl/8.18.0, and Accept: /*/*.

SOC relevance. A focused HTTP view links the user command to a specific endpoint, client port, server port, method, path, and request headers that can be used during timeline and session reconstruction.

Finding. The screenshot provides packet-level evidence of the controlled plaintext HTTP GET request. It is a companion view of the response in Figure 23 rather than a separate transaction.

Filtered HTTP 301 Redirect Response

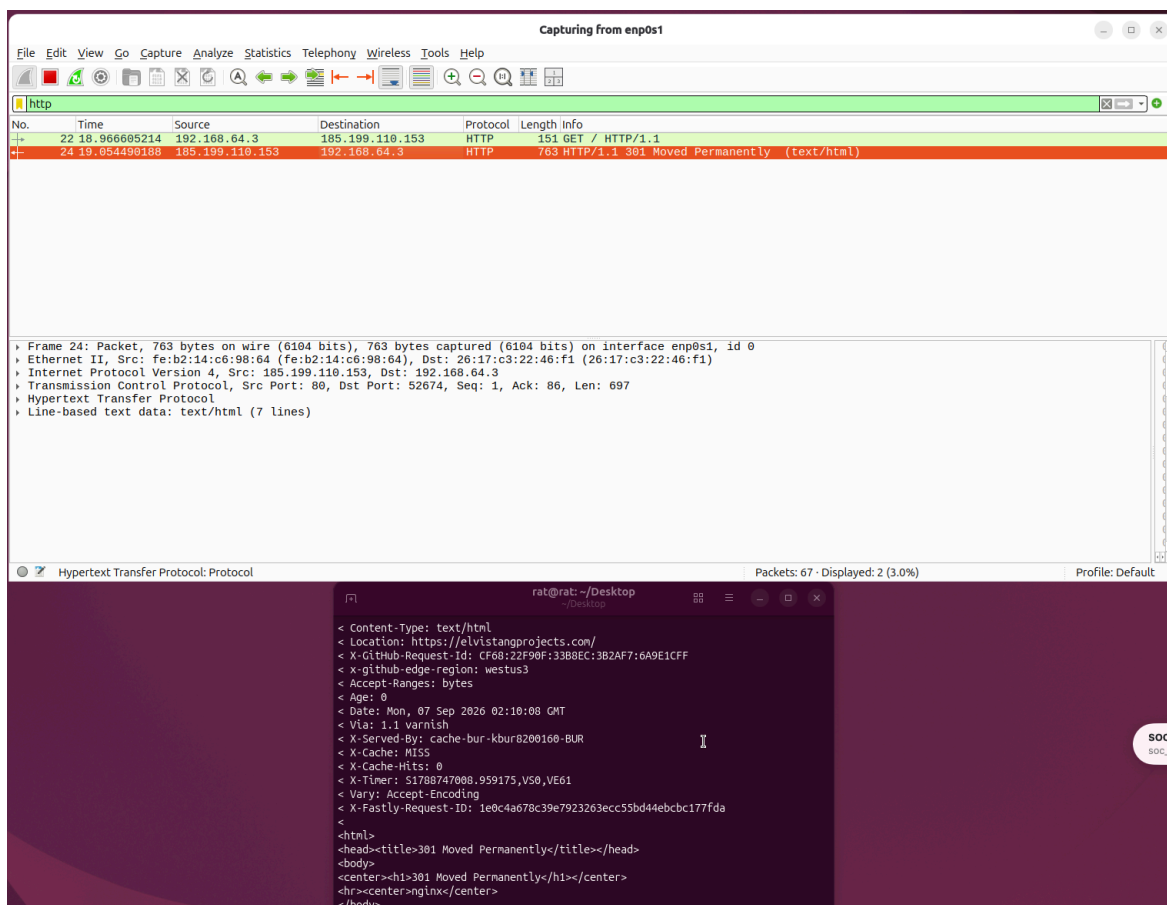


Figure 23 Wireshark HTTP filter with the 301 redirect response selected

Observation. Selected frame 24 is a 763-byte HTTP/1.1 301 Moved Permanently response from 185.199.110.153 to Ubuntu 192.168.64.3. Packet details show TCP source port 80, destination port 52674, acknowledgment 86, and 697 bytes of TCP payload. The visible curl output includes Connection: keep-alive, Content-Length: 162, Server: GitHub.com, Content-Type: text/html, and Location: <https://elvistangprojects.com/>. **SOC relevance.** Response status, direction, and redirect headers explain the transition from an initial plaintext request to a new HTTPS connection and help analysts correlate subsequent port 443 traffic. **Finding.** The exact HTTPS redirect target and server metadata are visible for the controlled request. Figures 22 and 23 are companion views of the same filtered HTTP exchange; the screenshot alone does not show whether the redirected TLS connection completed.

TLS 1.3 Client Hello

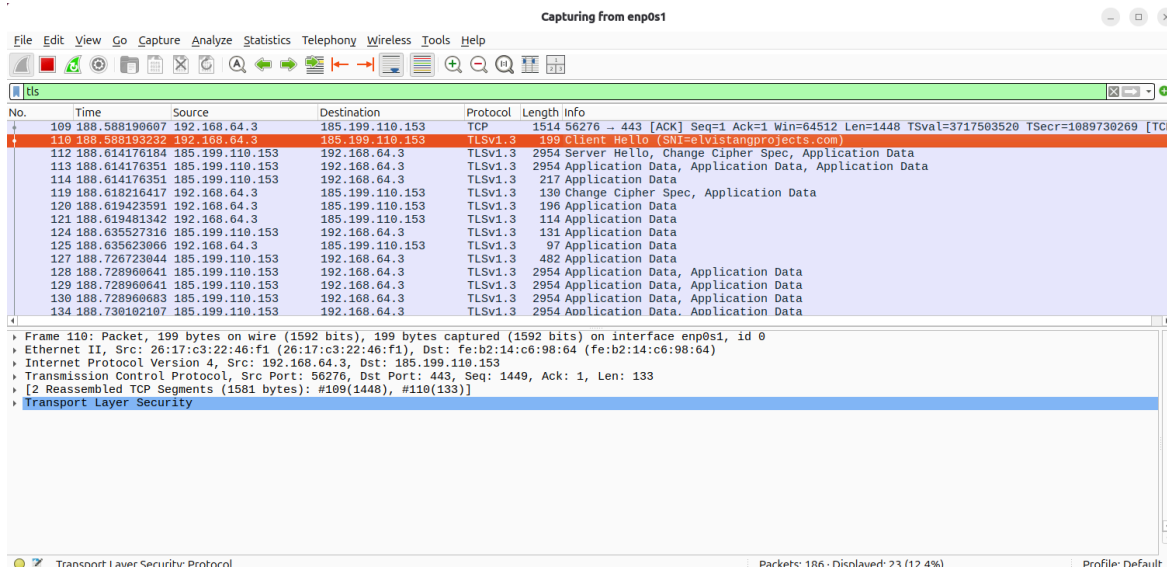


Figure 24 Wireshark TLS filter with the Client Hello selected

Observation. The tls filter shows a TLS 1.3 conversation between Ubuntu 192.168.64.3 and 185.199.110.153. Selected frame 110 is a 199-byte Client Hello from TCP source port 52676 to destination port 443, and the packet list displays SNI=elvistangprojects.com. The details also show 1,581 bytes reassembled from two TCP segments. The visible sequence continues with a Server Hello, Change Cipher Spec, and application-data packets in both directions. **SOC relevance.** The Client Hello identifies the initiating client, destination, service port, TLS version, and visible server name, providing useful connection metadata even when later payloads are encrypted. **Finding.** The client initiated a TLS 1.3 session to 185.199.110.153:443 with SNI elvistangprojects.com. The screenshot does not show certificate fields, negotiated cipher details, or application content.

TLS 1.3 Server Response

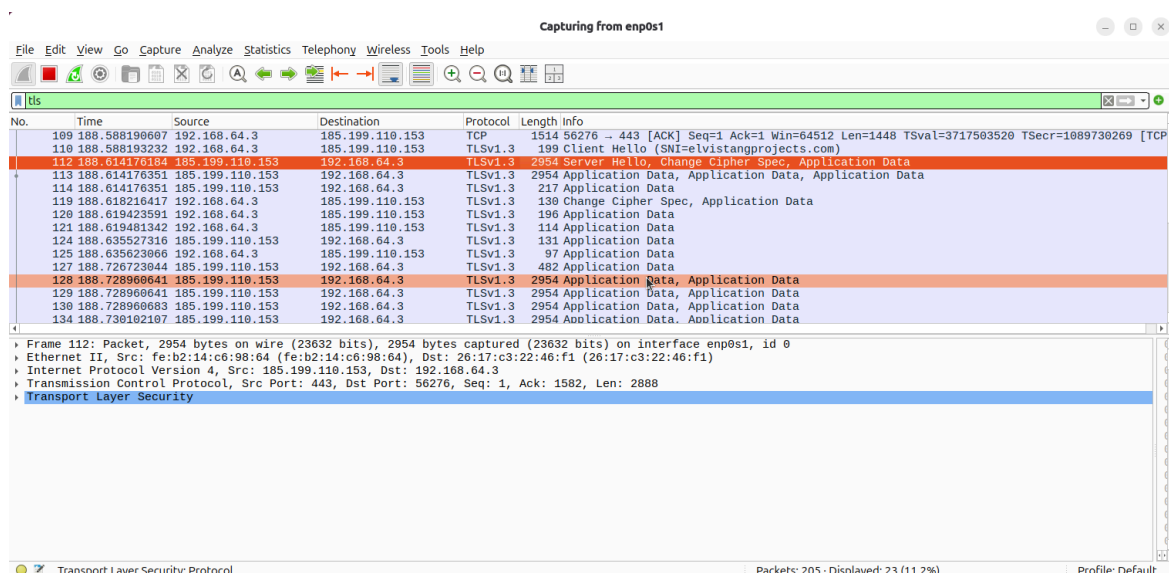


Figure 25 Wireshark TLS filter with the server response selected

Observation. Selected frame 112 is a 2,954-byte server-to-client packet from 185.199.110.153:443 to 192.168.64.3:52676. Wireshark labels the row TLSv1.3 Server Hello, Change Cipher Spec, Application Data. The TCP details show sequence 1, acknowledgment 1582, and 2,888 bytes of TCP payload. **SOC relevance.** The reverse direction and handshake labels corroborate that the remote endpoint responded to the Client Hello and that the session progressed into encrypted records. **Finding.** A TLS 1.3 server response is visible for the same endpoints and client port as Figure 24. The TLS tree is not expanded, so certificate identity, cipher suite, extensions, and validation outcome are not established by this screenshot.

TLS 1.3 Application Data

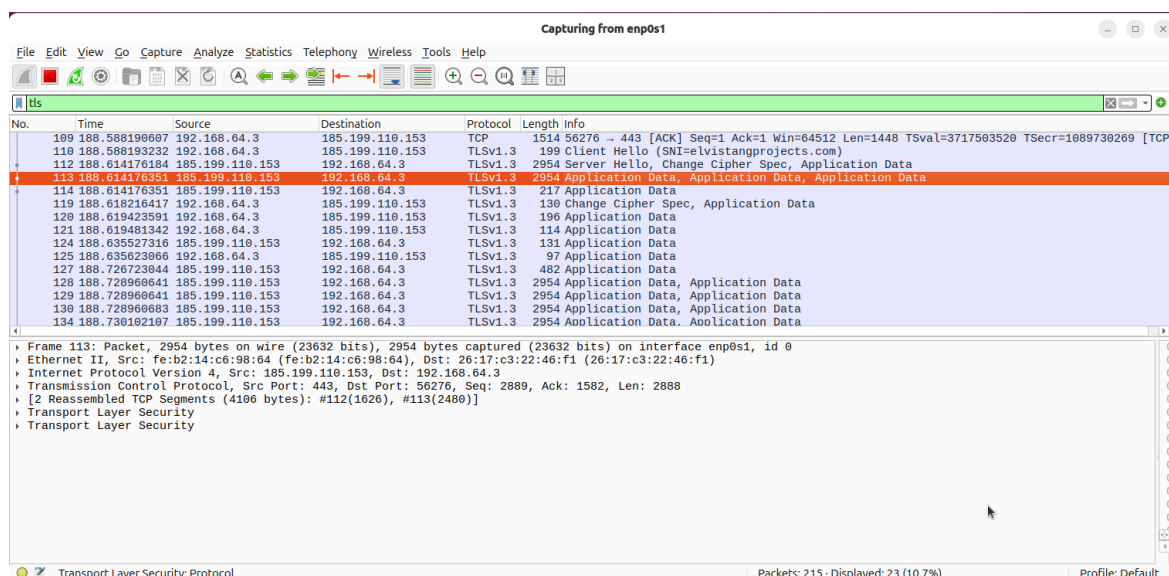


Figure 26 Wireshark TLS filter with encrypted application data selected

Observation. Selected frame 113 is a 2,954-byte server-to-client packet from 185.199.110.153:443 to Ubuntu 192.168.64.3:52676. The packet list labels it TLSv1.3 Application Data, Application Data. TCP details show sequence 2889, acknowledgment 1582, 2,888 bytes of TCP payload, and two reassembled segments totaling 4,106 bytes; two Transport Layer Security entries are visible in the packet tree. **SOC relevance.** Encrypted application records still reveal endpoints, direction, timing, sizes, and session continuity, which analysts can correlate with DNS, HTTP redirects, and endpoint telemetry without reading protected content. **Finding.** The TLS session carries encrypted application data from the server to Ubuntu. The screenshot does not reveal plaintext content or, by itself, determine whether the traffic is benign or malicious. Figures 24 through 26 are companion views of the same TLS conversation.

Supplemental Windows Endpoint and Scan Evidence

This evidence adds Windows connection-state and process context and records a scoped scan of the documented Windows endpoint. The screenshots support specific endpoint correlations but do not identify every process or establish malicious activity.

Windows TCP Connection Baseline

```

Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

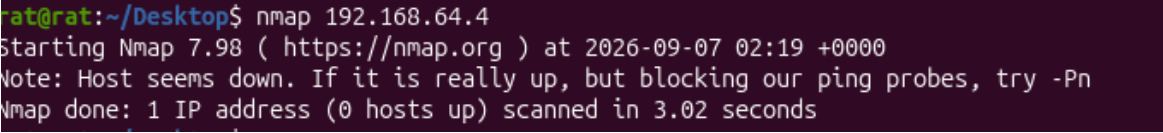
PS C:\Users\rat tester> Get-NetTCPConnection

LocalAddress      LocalPort RemoteAddress      RemotePort State      AppliedSetting
-----
:::               49672    :::               0          Listen
:::               49668    :::               0          Listen
:::               49667    :::               0          Listen
:::               49666    :::               0          Listen
:::               49665    :::               0          Listen
:::               49664    :::               0          Listen
:::               7680     :::               0          Listen
:::               445      :::               0          Listen
:::               135      :::               0          Listen
0.0.0.0           65215    0.0.0.0            0          Bound
0.0.0.0           57760    0.0.0.0            0          Bound
0.0.0.0           55716    0.0.0.0            0          Bound
0.0.0.0           54145    0.0.0.0            0          Bound
0.0.0.0           54143    0.0.0.0            0          Bound
0.0.0.0           49674    0.0.0.0            0          Bound
192.168.64.4      65215    4.153.187.221      443       CloseWait  Internet
192.168.64.4      57760    150.171.109.149    443       Established Internet
192.168.64.4      55716    52.152.143.207     443       CloseWait  Internet
192.168.64.4      54145    52.110.4.20        443       Established Internet
192.168.64.4      54143    52.110.4.20        443       Established Internet
192.168.64.4      49674    20.59.87.225       443       Established Internet
0.0.0.0           49672    0.0.0.0            0          Listen
0.0.0.0           49668    0.0.0.0            0          Listen
0.0.0.0           49667    0.0.0.0            0          Listen
0.0.0.0           49666    0.0.0.0            0          Listen
0.0.0.0           49665    0.0.0.0            0          Listen
0.0.0.0           49664    0.0.0.0            0          Listen
127.0.0.1         9843     0.0.0.0            0          Listen
0.0.0.0           5040     0.0.0.0            0          Listen
192.168.64.4      139      0.0.0.0            0          Listen
0.0.0.0           135      0.0.0.0            0          Listen
  
```

Figure 27 Windows TCP connection baseline from Get-NetTCPConnection

Observation. Get-NetTCPConnection shows listening and bound sockets plus active connections for Windows 192.168.64.4. Visible remote TCP port 443 sessions include established connections to 150.171.109.149, 52.110.4.20, and 20.59.87.225, while connections to 4.153.187.221 and 52.152.143.207 are in CloseWait. The output also shows local listeners including ports 135, 139, 445, 5040, 7680, 9843, and several dynamic ports. **SOC relevance.** A connection baseline separates listening services from outbound sessions and highlights states such as Established and CloseWait for follow-up. It also supplies endpoint and port data that can be correlated with packet captures. **Finding.** The Windows endpoint had multiple visible HTTPS connections and local listeners at the time shown. This default view does not display owning processes, application purpose, remote ownership, or whether any connection is suspicious.

Windows Host Discovery Scan



```
rat@rat:~/Desktop$ nmap 192.168.64.4
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-07 02:19 +0000
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.02 seconds
```

Figure 28 Nmap host-discovery result for Windows 192.168.64.4

Observation. Nmap 7.98 scans the documented Windows address 192.168.64.4 and reports that the host seems down. The scan completes in 3.02 seconds with zero hosts up and suggests using -Pn if the target blocks discovery probes. **SOC relevance.** A negative discovery result must be compared with known endpoint state and other evidence. Filtering, firewall behavior, or probe type can prevent discovery even when a host is running. **Finding.** Nmap received no response to its host-discovery probes in this attempt. Because Windows was documented at this address in other screenshots, this result alone does not establish that the endpoint was offline.

Windows Scan with Host Discovery Bypassed

```

fat@rat:~/Desktop$ nmap 192.168.64.4 -Pn
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-07 02:19 +0000
Packet Tracing enabled.
CONN (51.7057s) TCP localhost > 192.168.64.4:1048 => Operation now in progress
CONN (51.7063s) TCP localhost > 192.168.64.4:49167 => Operation now in progress
CONN (51.7065s) TCP localhost > 192.168.64.4:787 => Operation now in progress
CONN (51.7066s) TCP localhost > 192.168.64.4:1864 => Operation now in progress
CONN (51.7072s) TCP localhost > 192.168.64.4:1175 => Operation now in progress
CONN (51.7073s) TCP localhost > 192.168.64.4:5959 => Operation now in progress
CONN (51.7074s) TCP localhost > 192.168.64.4:20828 => Operation now in progress
CONN (51.7074s) TCP localhost > 192.168.64.4:33354 => Operation now in progress
CONN (51.7075s) TCP localhost > 192.168.64.4:5850 => Operation now in progress
CONN (51.7075s) TCP localhost > 192.168.64.4:9502 => Operation now in progress
CONN (52.7095s) TCP localhost > 192.168.64.4:9502 => Operation now in progress
CONN (52.7095s) TCP localhost > 192.168.64.4:5850 => Operation now in progress
CONN (52.7096s) TCP localhost > 192.168.64.4:33354 => Operation now in progress
CONN (52.7096s) TCP localhost > 192.168.64.4:20828 => Operation now in progress
CONN (52.7096s) TCP localhost > 192.168.64.4:5959 => Operation now in progress
CONN (52.7096s) TCP localhost > 192.168.64.4:1175 => Operation now in progress
CONN (52.7096s) TCP localhost > 192.168.64.4:1864 => Operation now in progress
CONN (52.7096s) TCP localhost > 192.168.64.4:787 => Operation now in progress
CONN (52.7096s) TCP localhost > 192.168.64.4:49167 => Operation now in progress
CONN (52.7097s) TCP localhost > 192.168.64.4:1048 => Operation now in progress
CONN (53.7155s) TCP localhost > 192.168.64.4:2100 => Operation now in progress
CONN (53.7155s) TCP localhost > 192.168.64.4:7025 => Operation now in progress
CONN (53.7155s) TCP localhost > 192.168.64.4:5679 => Operation now in progress
CONN (53.7156s) TCP localhost > 192.168.64.4:4002 => Operation now in progress
CONN (53.7156s) TCP localhost > 192.168.64.4:11967 => Operation now in progress
CONN (53.7156s) TCP localhost > 192.168.64.4:7625 => Operation now in progress
CONN (53.7156s) TCP localhost > 192.168.64.4:2399 => Operation now in progress
CONN (53.7156s) TCP localhost > 192.168.64.4:1783 => Operation now in progress
CONN (53.7156s) TCP localhost > 192.168.64.4:1075 => Operation now in progress
CONN (53.7156s) TCP localhost > 192.168.64.4:32778 => Operation now in progress
CONN (54.7192s) TCP localhost > 192.168.64.4:32778 => Operation now in progress
CONN (54.7193s) TCP localhost > 192.168.64.4:1075 => Operation now in progress

```

Figure 29 Nmap -Pn connection attempts to Windows 192.168.64.4

Observation. The command `nmap 192.168.64.4 -Pn` is started at 02:19 UTC with packet tracing enabled. The visible output shows repeated TCP connection attempts from localhost to many ports on 192.168.64.4, each labeled Operation now in progress. The screenshot is taken while the scan is still running and does not show the final Nmap port table. **SOC relevance.** The `-Pn` option bypasses host discovery and proceeds as though the target is online. A rapid series of connection attempts across many destination ports is the network pattern produced by a port scan and may be detected by endpoint or network monitoring. **Finding.** The scan progressed beyond discovery and attempted connections to multiple Windows TCP ports. No open, closed, or filtered port conclusion can be drawn because final scan results are not visible.

Filtered Established Windows Connections

```
PS C:\Users\rat tester> Get-NetTCPConnection | Where-Object State -eq "Established"
```

LocalAddress	LocalPort	RemoteAddress	RemotePort	State	AppliedSetting	OwningProcess
192.168.64.4	54145	52.110.4.20	443	Established	Internet	5888
192.168.64.4	54143	52.110.4.20	443	Established	Internet	5888
192.168.64.4	49674	20.59.87.225	443	Established	Internet	3524

```
PS C:\Users\rat tester> |
```

Figure 30 Established Windows TCP connections with owning process IDs

Observation. The PowerShell filter returns three Established connections from 192.168.64.4 to remote port 443. Two connections use local ports 54145 and 54143 to 52.110.4.20 and are owned by PID 5888. A third uses local port 49674 to 20.59.87.225 and is owned by PID 3524. **SOC relevance.** Filtering to established sessions reduces listener and transitional-state noise. The OwningProcess field creates a pivot from network indicators to the responsible endpoint process. **Finding.** Three active HTTPS connections are documented with local ports, remote IP addresses, and owning PIDs. Process names and remote-service identities are not established by this screenshot alone.

Expanded Established Connection Inventory

```

PS C:\Users\rat tester> Get-NetTCPConnection | Where-Object State -eq "Established" | Format-Table LocalAddress, LocalPort, RemoteAddress, RemotePort, State, OwningProcess -AutoSize

```

LocalAddress	LocalPort	RemoteAddress	RemotePort	State	OwningProcess
192.168.64.4	51440	150.171.27.10	443	Established	Internet
192.168.64.4	50214	150.171.27.10	443	Established	Internet
192.168.64.4	50018	150.171.109.151	443	Established	Internet
192.168.64.4	49674	20.59.87.225	443	Established	Internet
192.168.64.4	49492	150.171.109.151	443	Established	Internet

```

PS C:\Users\rat tester>

```

Figure 31 Expanded Windows established-connection inventory

Observation. The formatted PowerShell table shows multiple Established connections from 192.168.64.4 to remote TCP port 443. Visible owning PIDs are 8832, 6268, 5888, and 3524 across several remote addresses. PID 8832 appears repeatedly, while PID 6268 is associated with 23.66.3.137 and PID 3524 with 20.59.87.225 in the visible rows. **SOC relevance.** An expanded connection inventory helps identify repeated process activity, group sessions by owning PID, and select specific endpoints for DNS, reputation, packet, or process investigation. **Finding.** The screenshot records a broader set of active HTTPS sessions and their owning PIDs. It does not establish the names of PIDs 8832 or 5888, the ownership of the remote infrastructure, or whether the activity is expected.

Windows Process Correlation

```
PS C:\Users\rat tester> Get-Process -Id 6268
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
837	34	13428	79304	1.31	6268	1	Widgets

```
PS C:\Users\rat tester> Get-Process -Id 3524
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
411	19	5392	33992	0.38	3524	0	svchost

Figure 32 Process-name lookup for Windows connection owners

Observation. Get-Process -Id 6268 returns the process name Widgets, and Get-Process -Id 3524 returns svchost. When correlated with Figure 31, the visible connection owned by PID 6268 is to 23.66.3.137:443 and the connection owned by PID 3524 is to 20.59.87.225:443. **SOC relevance.** Process attribution strengthens network evidence by linking a remote connection to a local executable context. The generic svchost name still requires service-level or event-log evidence to identify the hosted Windows service. **Finding.** Two owning PIDs from the connection inventory are mapped to process names at the time shown. The evidence does not identify PIDs 8832 or 5888, establish process file paths or signatures, or determine whether either connection is benign or malicious.

Windows Security Audit Evidence

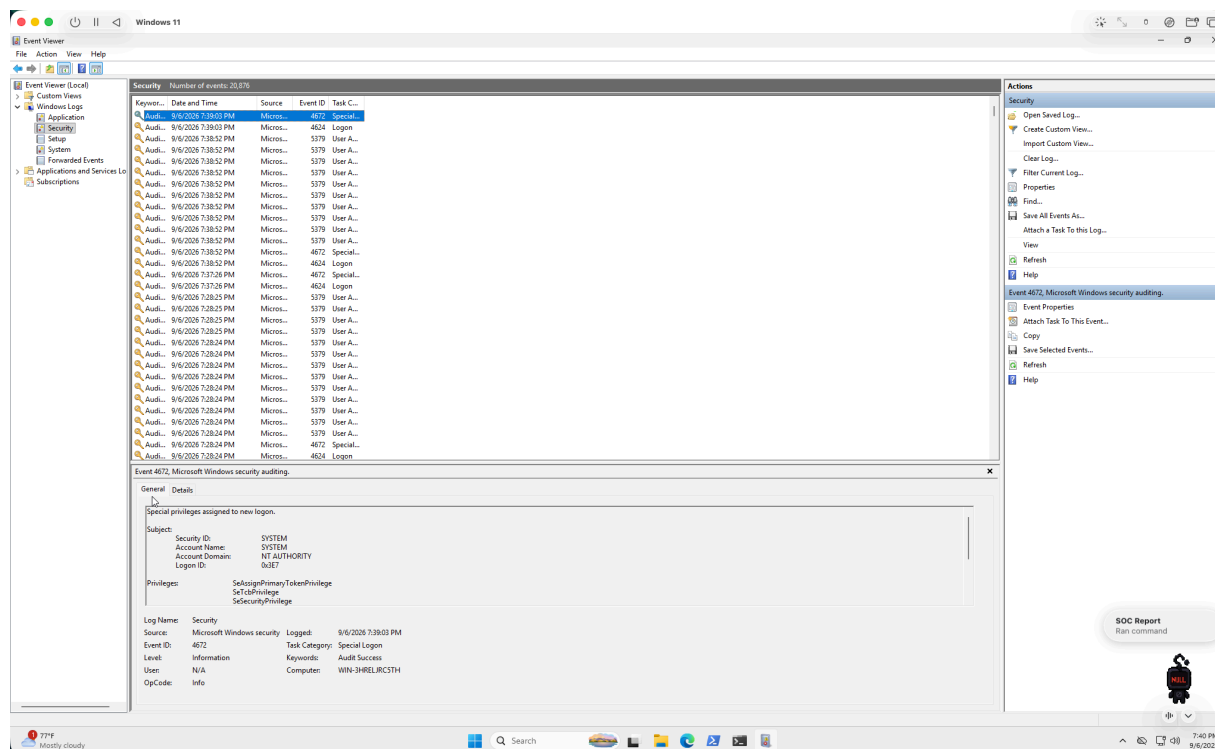


Figure 33 Windows Security Event ID 4672 selected in Event Viewer

Observation. Event Viewer is open to Windows Logs > Security, which shows 20,876 events. The selected entry is Microsoft Windows security auditing Event ID 4672 at 9/6/2026 7:39:03 PM. The General pane states Special privileges assigned to new logon and shows Security ID and Account Name SYSTEM, Account Domain NT AUTHORITY, Logon ID 0x3E7, and visible privileges including SeAssignPrimaryTokenPrivilege, SeTcbPrivilege, and SeSecurityPrivilege. The event is marked Information and Audit Success on computer WIN-3HRELJRC5TH.

SOC relevance. Event 4672 identifies a logon token receiving sensitive privileges. Analysts normally correlate its timestamp, account, and Logon ID with the related logon event and nearby process or service activity before deciding whether the behavior is expected.

Finding. The screenshot documents one successful privileged-logon audit event for the SYSTEM account. It does not show the complete privilege list, the initiating process, the related logon-event details, or evidence that the event is malicious.

Packet-Level View of the Windows Scan

No.	Time	Source	Destination	Protocol	Length	Info
2323	575.835449690	192.168.64.3	192.168.64.4	TCP	74	57574 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1928644384 TSecr=0 WS=1
2324	575.835455732	192.168.64.3	192.168.64.4	TCP	74	48114 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2836858198 TSecr=0 WS=1
2325	577.849276952	192.168.64.3	192.168.64.4	TCP	74	42648 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3553379882 TSecr=0 WS=1
2326	577.849489761	192.168.64.3	192.168.64.4	TCP	74	45454 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=805932597 TSecr=0 WS=10
2341	609.559889813	192.168.64.3	192.168.64.4	TCP	74	51046 → 8888 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3414016397 TSecr=0 WS=1
2342	609.559146185	192.168.64.3	192.168.64.4	TCP	74	35926 → 993 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3853610196 TSecr=0 WS=1
2343	609.559175921	192.168.64.3	192.168.64.4	TCP	74	48430 → 199 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3956921321 TSecr=0 WS=1
2344	609.559209604	192.168.64.3	192.168.64.4	TCP	74	33260 → 1025 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1371257261 TSecr=0 WS=1
2345	609.559229729	192.168.64.3	192.168.64.4	TCP	74	37392 → 25 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1990932320 TSecr=0 WS=1
2346	609.559258188	192.168.64.3	192.168.64.4	TCP	74	45650 → 25 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3099490274 TSecr=0 WS=1
2347	609.559267063	192.168.64.3	192.168.64.4	TCP	74	48760 → 256 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=420188584 TSecr=0 WS=1
2348	609.559290184	192.168.64.3	192.168.64.4	TCP	74	43774 → 995 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3949003270 TSecr=0 WS=1
2349	609.559310921	192.168.64.3	192.168.64.4	TCP	74	41938 → 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2870852365 TSecr=0 WS=1
2350	609.559336521	192.168.64.3	192.168.64.4	TCP	74	51554 → 110 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1685879892 TSecr=0 WS=1
2351	611.563659733	192.168.64.3	192.168.64.4	TCP	74	51566 → 110 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1684732330 TSecr=0 WS=1

Frame 2323: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface enp0s1, id 0
 Ethernet II, Src: 26:17:c3:22:46:f1 (26:17:c3:22:46:f1), Dst: 0a:66:a0:9e:86:03 (0a:66:a0:9e:86:03)
 Internet Protocol Version 4, Src: 192.168.64.3, Dst: 192.168.64.4
 Transmission Control Protocol, Src Port: 57574, Dst Port: 80, Seq: 0, Len: 0

Figure 34 Wireshark SYN traffic from Ubuntu to the Windows endpoint

Observation. The accepted display filter `ip.addr == 192.168.64.4 && tcp` shows TCP traffic involving the Windows endpoint. Visible rows are 74-byte SYN packets from Ubuntu 192.168.64.3 to Windows 192.168.64.4 across destination ports including 80, 443, 8888, 993, 199, 1025, 25, 256, 995, 53, and 110. Selected frame 2323 uses source port 57574 and destination port 80 with sequence 0 and no payload. The status bar shows 7,619 packets captured and 4,809 displayed. **SOC relevance.** A burst of SYN packets from one source to many destination ports is packet-level evidence of port-scanning behavior. Correlating source, destination, timestamps, and flags connects the Nmap command-line activity to observable network telemetry. **Finding.** The capture corroborates that Ubuntu generated TCP SYN probes targeting multiple ports on Windows during the authorized lab scan. No SYN-ACK or reset response is visible in the displayed rows, so these rows alone do not establish individual port states.

Completed Windows -Pn Scan Result

```

rat@rat:~/Desktop$ nmap -Pn -sV 192.168.64.4
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-07 02:22 +0000
Nmap scan report for 192.168.64.4
Host is up.
All 1000 scanned ports on 192.168.64.4 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 202.39 seconds
rat@rat:~/Desktop$ nmap -Pn 192.168.64.4
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-07 02:36 +0000
Nmap scan report for 192.168.64.4
Host is up.
All 1000 scanned ports on 192.168.64.4 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 202.23 seconds
rat@rat:~/Desktop$

```

Figure 35 Completed Nmap -Pn results for Windows 192.168.64.4

Observation. The nmap -Pn -sV 192.168.64.4 scan starts at 02:22 UTC and completes in 202.39 seconds; the later nmap -Pn 192.168.64.4 scan starts at 02:36 UTC and completes in 202.23 seconds. Both outputs report the target as up and state that all 1,000 scanned TCP ports are filtered with no response. The service-detection run lists no identified services. **SOC relevance.** A filtered/no-response result indicates that probes did not receive the replies needed to classify ports as open or closed. Because -Pn skips normal host discovery and treats the target as online for scanning, the host-up label should be interpreted with that option in mind and correlated with independent endpoint evidence. **Finding.** The completed scans produced no open-port evidence for Windows and classified all 1,000 scanned TCP ports as filtered because no responses were received. This supports filtering or dropped-probe behavior but does not prove that no services are listening.

Observations and Findings

Thirty-four screenshots have been reviewed: twenty-seven from Ubuntu and seven from Windows. They document analyst tooling, both virtual-machine addresses, Shared NAT routing, command-line and packet-level reachability, DNS query-response activity, TCP and plaintext HTTP behavior, an exact HTTPS redirect, TLS 1.3 session metadata, Ubuntu sockets, command-line packet capture, Windows connection state, limited process attribution, a selected Windows Security audit event, and command-line plus packet-level scan activity. Controlled Windows-generated application traffic and additional correlated event-log evidence have not yet been supplied; two connection-owner PIDs also remain unidentified.

Record type	How it will be written	Example form
Observation	A verifiable fact visible in the screenshot or packet details.	The capture shows a DNS query from the documented client to the documented resolver.
Interpretation	A technical explanation of what the visible fields indicate.	The query requests name resolution before the client initiates a connection.

Record type	How it will be written	Example form
Finding	A supported conclusion that answers the investigation objective.	The observed lookup and subsequent connection are consistent with the controlled test activity.
Limitation	A boundary that prevents a stronger conclusion.	The encrypted payload is not visible, so content-level behavior cannot be verified from this capture alone.

Findings Summary

Current status: Ubuntu and Windows network baselines, command-line reachability, Wireshark ICMP, DNS, TCP, HTTP, TLS, and Nmap scan evidence, Windows connection and limited process data, and one Windows Security audit event have been reviewed; controlled Windows-generated application traffic and additional event correlation remain pending.

Ubuntu is configured as 192.168.64.3/24 and Windows as 192.168.64.4/24, both using the Shared NAT gateway 192.168.64.1. Windows reached Ubuntu with four ICMP replies and 0% loss. Wireshark on enp0s1 isolated gateway ICMP request-reply pairs, DNS query-response pairs, and a controlled HTTP request to 185.199.110.153:80 followed by a visible 301 redirect to <https://elvistangprojects.com/>. A subsequent TLS 1.3 conversation to 185.199.110.153:443 includes the Client Hello SNI, server response, and encrypted application data. Windows connection evidence shows multiple established outbound sessions to TCP port 443 and supplies owning PIDs; two PIDs are correlated to Widgets and svchost, while two remain unidentified. Event Viewer records Security Event ID 4672 for a SYSTEM logon receiving special privileges, but the screenshot alone does not establish malicious activity. Standard Nmap discovery received no response from Windows; packet capture later showed the authorized -Pn SYN probes, and completed scans classified all 1,000 scanned TCP ports as filtered due to no response. The gateway scan reported ports 53, 5000, and 7000 open, but port labels do not verify the running software. No supplied screenshot independently establishes malicious activity.

Lessons Learned

The evidence demonstrates why analysts must verify interface identity, addressing, routes, and the exact target before interpreting network activity. Protocol filters reduced background traffic to focused ICMP, DNS, TCP, HTTP, TLS, and scan views. The HTTP request and redirect connect a user action to a specific endpoint and HTTPS destination, while the TLS views show how SNI, endpoints, ports, direction, and record sizes remain useful after encryption begins. Windows connection filtering adds endpoint context through owning PIDs, targeted lookups show both the value and limits of process attribution, and Event ID 4672 shows why privileged-logon events require correlation rather than an immediate malicious conclusion. Comparing failed discovery, visible SYN probes, and completed filtered results demonstrates how command output, packet evidence, and endpoint context strengthen one another.

- How NAT and interface selection affected traffic visibility.
- Which display filters most effectively reduced noise without hiding relevant context.
- How packet fields connected a user action to network behavior.
- Where endpoint information strengthened or limited the packet analysis.

- What would be changed in a second investigation to improve collection and reporting.

Portfolio and Interview Talking Points

- Explain the lab design and why Shared NAT was selected for the initial exercise.
- Walk through one evidence item from command to packet capture to conclusion.
- Describe how facts, interpretation, and findings were kept separate.
- Discuss one visibility limitation and the additional telemetry that would address it.

Appendix A Command Reference

Ubuntu

```
ip addr
ip route
ping <destination>
nslookup <domain>
curl -I <url>
ss -tupn
sudo tcpdump -i <interface>
nmap <authorized lab host>
nmap -Pn <authorized lab host>
nmap -Pn -sV <authorized lab host>
```

Windows PowerShell

```
ipconfig /all
ping <Ubuntu IP>
nslookup <domain>
netstat -ano
Get-NetTCPConnection
Get-NetTCPConnection | Where-Object State -eq "Established"
Get-Process -Id <PID>
```

Values shown in angle brackets are inputs to be replaced with observed, authorized lab targets. They are not findings or confirmed environment values.

Document Control

Version	Date	Status	Change
1.0	September 6 2026	In progress	Created portfolio report structure and evidence plan
1.1	September 6 2026	In progress	Added Ubuntu setup and network baseline evidence
1.2	September 6 2026	In progress	Added DNS, HTTP, socket, tcpdump, and Nmap evidence
1.3	September 6 2026	In progress	Added Windows addressing, VM reachability, and Shared NAT gateway scan evidence
1.4	September 6 2026	In progress	Added Wireshark interface context and filtered ICMP request and reply evidence
1.5	September 6 2026	In progress	Added Wireshark DNS query-response and TCP/HTTP redirect evidence; refreshed findings and evidence status
1.6	September 6 2026	In progress	Added focused HTTP request and response plus TLS 1.3 handshake and encrypted application-data evidence
1.7	September 6 2026	In progress	Added Windows connection and process correlation plus Windows scan evidence
1.8	September 6 2026	In progress	Added Windows Security event evidence and packet-level plus completed Windows scan results